

The DBA Arcade: Quest for High Availability

Handover de Automatización y Monitoreo
Proactivo – Base de Datos Grados Titulos

 PULSE START PARA CONTINUAR

24 de Octubre de 2023 | Equipo de Arquitectura de Datos

Arquitectura de Entorno y Reglas del Motor

Contexto Técnico

Motor de Base de Datos:	"SQL Server 2021 (v16.x)"
Base de Datos:	"GradosTitulos"
Esquemas:	"Academico, Tramite, Catalogo, Evaluacion, Documento"
Estrategia:	"Automatización desatendida vía SQL Agent y PowerShell"

El Tech Stack



T-SQL (Core Engine): Lógica transaccional e integridad de datos.



SQL Server Agent (Task Master): Orquestación temporal y alertas.



PowerShell (File Exfiltrator): Gestión de archivos y exfiltración de reportes externos.

Misión: Tema 1, Proyecto 1

La Oficina de Grados y Títulos requiere que todos los días a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.



Justificación

Se utiliza SQL Server Agent para centralizar la lógica de negocio temporal directamente en el motor, evitando dependencias de aplicaciones externas de terceros.



Buenas Prácticas

Segmentar los pasos de los Jobs para identificar puntos exactos de ruptura.

```
USE GradosTitulos;
GO
EXEC msdb.dbo.sp_add_job @job_name =
N'Reporte_Diario_Tramites_Registrados';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'Reporte_Diario_Tramites_Registrados',
    @step_name = N'Contar_Tramites_Ayer',
    @subsystem = N'TSQL',
    @command = N'
DECLARE @Cant INT;
SELECT @Cant = COUNT(*) FROM Tramite.Tramite
WHERE FechaInicio = CAST(DATEADD(day, -1,
SYSDATETIME()) AS DATE);
INSERT INTO dbo.AuditoriaJobs (NombreJob, Resultado)
VALUES ('Reporte_Diario_Tramites', 'Total ayer: '
+ CAST(@Cant AS VARCHAR(10)));';
    @database_name = N'GradosTitulos';

EXEC msdb.dbo.sp_add_jobschedule
    @job_name = N'Reporte_Diario_Tramites_Registrados',
    @name = N'Schedule_Diario_08AM',
    @freq_type = 4, @freq_interval = 1,
    @active_start_time = 080000;
EXEC msdb.dbo.sp_add_jobserver @job_name =
N'Reporte_Diario_Tramites_Registrados';
```


Misión: Tema 1, Proyecto 2

Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado EN_PROCESO, registrando la ejecución.



Justificación

Automatizar el rastreo de estados reduce drásticamente el overhead manual del DBA y asegura que las métricas de latencia de los trámites sean siempre precisas.



Buenas Prácticas

Usar SYSDATETIME() en lugar de GETDATE() para obtener mayor precisión de auditoría nativa en SQL Server 2021.

```
EXEC msdb.dbo.sp_add_job @job_name = N'Actualizacion_Semanal_Tramites_EnProceso';
```

```
EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'Actualizacion_Semanal_Tramites_EnProceso',
    @step_name = N'Update_FechaUltMod_Proceso',
    @subsystem = N'TSQL',
    @command = N'
UPDATE T SET T.FechaUltMod = SYSDATETIME()
FROM Trámite.Tramite T
INNER JOIN Catalogo.EstadoTrámite E ON T.IdEstadoTrámite = E.IdEstadoTrámite
WHERE E.Descripcion = ''EN_PROCESO'';
```

```
INSERT INTO dbo.AuditoriaJobs (NombreJob, Resultado)
VALUES ('Actualizacion_Semanal_Tramites_EnProceso',
        'Filas actualizadas: ' + CAST(@@ROWCOUNT AS VARCHAR(10)));',
    @database_name = N'GradosTitulos';
```

```
EXEC msdb.dbo.sp_add_jobschedule
    @job_name = N'Actualizacion_Semanal_Tramites_EnProceso',
    @name = N'Schedule_Lunes_01AM',
    @freq_type = 8, @freq_interval = 2, @active_start_time = 010000;
```

```
EXEC msdb.dbo.sp_add_jobserver @job_name =
N'Actualizacion_Semanal_Tramites_EnProceso';
```


Misión: Tema 1, Proyecto 3

Diseñar un Job que elimine mensualmente los registros temporales de **Tramite.LogTemporal**. Registros > 180 días deben moverse a una tabla histórica y luego eliminarse.

Justificación



Previene la degradación de rendimiento en tablas temporales. El uso de transacciones explícitas asegura que ninguna fila se elimine si la inserción histórica falla (Atomicidad).

Buenas Prácticas



Habilitar SET XACT_ABORT ON para garantizar el rollback automático en caso de errores de constraint o interrupciones en tiempo de ejecución.

```
SET XACT_ABORT ON;
```

```
BEGIN TRY
```

```
    BEGIN TRANSACTION;
```

```
    INSERT INTO Tramite.LogHistorico (IdLog, Detalle, Fecha)
    SELECT IdLog, Detalle, Fecha FROM Tramite.LogTemporal
    WHERE Fecha < DATEADD(DAY, -180, SYSDATETIME());
```

```
    DELETE FROM Tramite.LogTemporal \
    WHERE Fecha < DATEADD(DAY, -180, SYSDATETIME());
```

```
    COMMIT TRANSACTION;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
```

```
    DECLARE @Err NVARCHAR(MAX) = ERROR_MESSAGE();
```

```
    INSERT INTO dbo.AuditoriaJobs (NombreJob, Resultado) \
    VALUES ('Depuracion_Mensual_Logs', 'Fallo: ' + @Err);
    THROW;
```

```
END CATCH;
```


Misión: Tema 1, Proyecto 4

Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a 17 durante el registro de trámites.

Justificación

El uso de severidades ≥ 17 en alertas es vital, ya que estos niveles indican problemas que el usuario no puede corregir (recursos agotados o inconsistencias), requiriendo intervención inmediata del DBA.

Buenas Prácticas

Configurar `@delay_between_responses = 300` para evitar la inundación de correos electrónicos (mail flooding) durante incidentes sistémicos.

-- Configurar Operador para recibir notificaciones

```
EXEC msdb.dbo.sp_add_operator \  
    @name = N'DBA_Senior_UPLA', \  
    @enabled = 1, \  
    @email_address = N'soporte_db@upla.edu.pe';
```

-- Alerta para Severidades Críticas (17: Insuficiencia a 25: Fatal)

```
EXEC msdb.dbo.sp_add_alert \  
    @name = N'Alert_Severidad_Critica_GT', \  
    @message_id = 0, \  
    @severity = 17, \  
    @enabled = 1, \  
    @delay_between_responses = 300, \  
    @include_event_description_in = 1, \  
    @database_name = N'GradosTitulos';
```

```
EXEC msdb.dbo.sp_add_notification \  
    @alert_name = N'Alert_Severidad_Critica_GT', \  
    @operator_name = N'DBA_Senior_UPLA', \  
    @notification_method = 1;
```


Misión: Tema 1, Proyecto 5

Sistema automático diario: verificar trámites pendientes > 30 días, generar reporte, registrar ejecución, y enviar correo mediante Database Mail.

Justificación

Transforma la base de datos de un repositorio pasivo a un sistema proactivo que empuja información crítica directamente a los tomadores de decisiones.

Buenas Prácticas

Retener el historial de Jobs por al menos 30 días para análisis retrospectivo de tendencias de falla.

```
DECLARE @Count INT;
SELECT @Count = COUNT(*) FROM Tramite.Tramite
WHERE IdEstadoTramite NOT IN (5, 6, 7)
AND DATEDIFF(DAY, FechaInicio, SYSDATETIME()) > 30;

IF @Count > 0
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'Perfil_Notificaciones',
        @recipients = 'coordinador_gt@upla.edu.pe',
        @subject = 'ALERTA: Trámites con retraso crítico',
        @body = 'Se han detectado ' + CAST(@Count AS
    VARCHAR) + ' trámites con más de 30 días sin
    finalizar.';

    INSERT INTO dbo.AuditoriaJobs (NombreJob, Resultado)
    VALUES ('Monitor_Tramites_Antiguos',
        'Alerta enviada para ' + CAST(@Count AS
    VARCHAR) + ' registros. ');
END
```


Misión: Tema 2, Proyecto 1

Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de GradosTitulos. Incluir: BACKUP, compresión, verificación y programación.

Justificación

La capa base de la recuperación ante desastres. La compresión nativa reduce drásticamente el cuello de botella de I/O en disco durante la ventana de mantenimiento.

Buenas Prácticas

Incluir siempre la cláusula CHECKSUM para validar la integridad de las páginas de datos en el momento de la lectura, detectando corrupción temprana.

```
DECLARE @BackupName NVARCHAR(200) =  
    N'D:\Backups\GradosTitulos_Full_' +  
    REPLACE(CONVERT(NVARCHAR, SYSDATETIME(), 112),  
    ':',  
    ':', '' ) +  
    '.bak';
```

```
BACKUP DATABASE [GradosTitulos]  
TO DISK = @BackupName  
WITH COMPRESSION,  
CHECKSUM,  
STATS = 10;
```


Misión: Tema 2, Proyecto 2

Crear un plan semanal para reorganizar índices de las tablas clave: Persona, Tramite, TrabajoInvestigacion, Sustentacion.



Justificación

REORGANIZE es una operación de bajo impacto que desfragmenta el nivel de hoja sin emitir bloqueos exclusivos a nivel de tabla, preservando la concurrencia.



Buenas Prácticas

Ejecutar operaciones de mantenimiento de índices en ventanas de baja carga transaccional para minimizar la presión sobre el Transaction Log.

-- Mantenimiento Semanal de Índices de Bajo Impacto

```
ALTER INDEX ALL ON Academico.Persona  
REORGANIZE;
```

```
ALTER INDEX ALL ON Tramite.Tramite  
REORGANIZE;
```

```
ALTER INDEX ALL ON  
Tramite.TrabajoInvestigacion  
REORGANIZE;
```

```
ALTER INDEX ALL ON  
Evaluacion.Sustentacion REORGANIZE;
```


Misión: Tema 2, Proyecto 4

Automatizar el mantenimiento completo: Backup, CHECKDB, limpieza de historial, reorganización y actualización de estadísticas.



Justificación

Una rutina de higiene completa garantiza la validación de la estructura física (CHECKDB) y asegura que el optimizador de consultas disponga del histograma más reciente para crear planes eficientes.



Buenas Prácticas

En secuencias automatizadas, ejecutar DBCC CHECKDB antes del Backup. Esto evita respaldar silenciosamente una base de datos corrupta.

```
-- 1. Validación de Integridad Estructural y Lógica  
DBCC CHECKDB([GradosTitulos]) WITH NO_INFOMSGS;
```

```
-- 2. Limpieza de artefactos históricos de MSDB  
EXEC msdb.dbo.sp_purge_jobhistory @oldest_date =  
'2026-01-01';
```

```
-- 3. Actualización profunda del histograma  
UPDATE STATISTICS Tramite.Tramite WITH FULLSCAN;
```

```
-- NOTA: El Backup Full se ejecuta como paso posterior  
-- tras validar que CHECKDB ha retornado limpio.
```


Misión: Tema 2, Proyectos 3 y 5

Implementar un plan inteligente que determine automáticamente qué operaciones ejecutar (Reorganize vs Rebuild) según el nivel de fragmentación encontrado en sys.dm_db_index_physical_stats.



Justificación

El mantenimiento inteligente ahorra inmensas cantidades de I/O y espacio en el Transaction Log al reconstruir únicamente los objetos que verdaderamente lo necesitan.



Buenas Prácticas

Omitir el mantenimiento en tablas pequeñas (ej. < 1000 páginas) donde la fragmentación física no degrada el rendimiento de lectura.

```
DECLARE cur_index CURSOR FOR
SELECT OBJECT_NAME(s.object_id), i.name, s.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
'LIMITED') s
JOIN sys.indexes i ON s.object_id = i.object_id AND
s.index_id = i.index_id
WHERE s.avg_fragmentation_in_percent > 10 AND i.name IS NOT NULL;
```

```
OPEN cur_index;
```

```
FETCH NEXT FROM cur_index INTO @TableName, @IndexName, @Frag;
WHILE @@FETCH_STATUS = 0
BEGIN
    IF @Frag BETWEEN 10 AND 30
        SET @SQL = 'ALTER INDEX ' + QUOTENAME(@IndexName) + ' ON ' +
        QUOTENAME(@TableName) + ' REORGANIZE;';
    ELSE
        SET @SQL = 'ALTER INDEX ' + QUOTENAME(@IndexName) + ' ON ' +
        QUOTENAME(@TableName) + ' REBUILD WITH (ONLINE = ON);';
    EXEC (@SQL);

    SET @SQL = 'UPDATE STATISTICS ' + QUOTENAME(@TableName) + ';';
    EXEC (@SQL);

    FETCH NEXT FROM cur_index INTO @TableName, @IndexName, @Frag;
END

CLOSE cur_index;
DEALLOCATE cur_index;
```


Misión: Tema 3, Proyecto 1

Crear un procedimiento almacenado que genere automáticamente un respaldo completo, diseñado para ser invocado posteriormente desde PowerShell.



Justificación

Encapsular la lógica del respaldo dentro de un Stored Procedure de T-SQL protege la sintaxis exacta y los parámetros de compresión, otorgando a PowerShell únicamente permisos de ejecución.



Buenas Prácticas

Mantener un directorio estándar y aislado para los respaldos automáticos, asegurando políticas de retención claras a nivel de sistema operativo.

```
CREATE OR ALTER PROCEDURE dbo.SP_BackupInstitucional
AS
BEGIN
    SET NOCOUNT ON;

    -- Definición de ruta estandarizada y segura
    DECLARE @Path NVARCHAR(255) = 'D:\Backups\Backup_Auto.bak';

    -- Ejecución de respaldo empaquetado para invocación externa
    BACKUP DATABASE GradosTitulos
    TO DISK = @Path
    WITH INIT,
        COMPRESSION,
        STATS = 5;

    -- El control de errores será manejado por PowerShell
    -- mediante el flag -ErrorAction Stop
END;
```


Misión: Tema 3, Proyecto 2

Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.



Justificación

PowerShell maneja la exfiltración y el formateo de datos fuera del motor de SQL Server, ahorrando valiosos ciclos de CPU del servidor para las transacciones críticas.



Buenas Prácticas

Utilizar siempre el flag **-NoTypeInfo** y especificar explícitamente **-Encoding UTF8** para evitar corrupción de caracteres especiales en sistemas externos.

```
$query = "SELECT * FROM Tramite.Tramite  
WHERE IdEstadoTramite = 5"
```

```
Invoke-Sqlcmd -ServerInstance "." -  
Database "GradosTitulos" -Query  
$query |
```

```
    Export-Csv -Path  
    "D:\Reportes\Aprobados.csv" ` -  
        -NoTypeInfo ` -  
        -Encoding UTF8
```

```
Write-Host ""Exfiltración completada y  
formateada correctamente.""
```


Misión: Tema 3, Proyecto 3

Consultar diariamente la cantidad de trámites y generar un archivo Excel, almacenando una copia con sello de tiempo.



Justificación

Permite la generación directa de artefactos listos para el negocio, eliminando la necesidad de aplicaciones intermediarias o costosos pipelines de ETL para reportes simples.



Buenas Prácticas

Envolver los comandos de exportación en bloques try/catch para manejar elegantemente errores de bloqueo de archivos a nivel de sistema operativo.

```
# Requisito: Install-Module -Name ImportExcel -Scope
CurrentUser

try {
    $sqlQuery = "SELECT COUNT(*) as Total FROM
Tramite.Tramite"

    $data = Invoke-Sqlcmd -ServerInstance "." `
        -Database "GradosTitulos" `
        -Query $sqlQuery

    $dynamicPath =
"D:\Backups\Reporte_Tramites_$((Get-Date).ToString
('yyyyMMdd')).xlsx"

    $data | Export-Excel -Path $dynamicPath

} catch {
    Write-Error "Falla en generación Excel:
$($_.Exception.Message)"
}
```


Misión: Tema 3, Proyecto 4

Sistema automático para detectar docentes inactivos (> 1 año) preparando los datos para su consumo en reportes.



Justificación

El uso de una Vista encapsula la compleja lógica de Anti-Semi-Join, presentando un dataset limpio y plano para que PowerShell o la capa de reportes lo consuma sin procesar lógica de negocio.



Buenas Prácticas

Priorizar NOT EXISTS sobre NOT IN al filtrar contra subconsultas, ya que maneja los valores NULL de manera mucho más eficiente en el plan de ejecución.

```
CREATE OR ALTER VIEW Academico.V_DocentesInactivos AS
CREATE OR ALTER VIEW Academico.V_DocentesInactivos AS
SELECT p.Nombres, p.ApellidoPaterno
FROM Academico.Docente d
    JOIN Academico.Persona p ON d.IdPersona = p.IdPersona

WHERE d.EstadoActivo = 0
    OR NOT EXISTS (
        SELECT 1
        FROM Tramite.DesignacionAsesor da
        WHERE da.IdDocente = d.IdDocente
        AND da.FechaDesignacion > DATEADD(YEAR, -1, SYSDATETIME())
    );
```

-- Esta vista presenta una superficie de ataque plana
-- lista para ser exportada vía Invoke-Sqlcmd en PowerShell.

Misión: Tema 3, Proyecto 5

Solución integral: ejecutar respaldos, validar integridad (CHECKDB), comprimir a ZIP, eliminar archivos mayores a 30 días, registrar auditoría en T-SQL.

Justificación

Crea un ciclo de mantenimiento totalmente autosuficiente. La compresión a ZIP post-respaldo ahorra almacenamiento a largo plazo sin cargar el motor SQL.

Buenas Prácticas

Escribir los fallos críticos del script directamente en la tabla de auditoría interna de SQL Server, garantizando un monitoreo centralizado para el DBA.

CHECKDB

BACKUP

COMPRESIÓN ZIP

PURGA 30 DÍAS

```
$Server = "."; $DB = "GradosTitulos"; $BackupDir = "D:\Backups"
$Date = Get-Date -Format "yyyyMMdd_HHmm"
$BakFile = "$BackupDir\$DB`_$Date.bak"; $ZipFile = "$BackupDir\$DB`_$Date.zip"

try {
    $sql = "DBCC CHECKDB([$DB]) WITH NO_INFOMSGS;
           BACKUP DATABASE [$DB] TO DISK = '$BakFile' WITH COMPRESSION;"
    Invoke-Sqlcmd -ServerInstance $Server -Query $sql -ErrorAction Stop

    if (Test-Path $BakFile) {
        Compress-Archive -Path $BakFile -DestinationPath $ZipFile -Force
        Remove-Item $BakFile
    }

    Get-ChildItem $BackupDir -Filter "*.zip" |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-30) } | Remove-Item
} catch {
    $msg = "Error crítico: $($_.Exception.Message)"
    Invoke-Sqlcmd -ServerInstance $Server -Query "INSERT INTO dbo.AuditoriaJobs
           (NombreJob, Resultado) VALUES ('PS_Maint', '$msg')"
}
```


Misión: Tema 4, Proyecto 1

Crear un procedimiento almacenado que consulte la cantidad de trámites pendientes (< Estado 5) y envíe un correo resumen al DBA.



Justificación

Mantener al equipo de TI al tanto del volumen de carga pendiente en el sistema permite anticipar picos de concurrencia o posibles fallos en el flujo del usuario.



Buenas Prácticas

Configurar perfiles de Database Mail dedicados (Perfil_UPLA) para aislar los correos de sistema de las comunicaciones de marketing o aplicación.

```
CREATE OR ALTER PROCEDURE Tramite.SP_AlertaPendientes
AS
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @Total INT = (
        SELECT COUNT(*)
        FROM Tramite.Tramite
        WHERE IdEstadoTramite < 5
    );
```

```
    DECLARE @Body NVARCHAR(MAX) =
        'Se reportan ' + CAST(@Total AS VARCHAR) + ' trámites
        en estado pendiente.';
```

```
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'Perfil_UPLA',
        @recipients = 'dba@upla.edu.pe',
        @subject = 'Resumen Pendientes Operativos',
        @body = @Body;
```

```
END;
```


Misión: Tema 4, Proyecto 2

Registrar automáticamente en una tabla de auditoría todos los errores en SP_RegistrarTramite, aplicando validación de negocio Senior.



Justificación

El manejo centralizado previene que excepciones no controladas rompan las capas de la aplicación. Validar el negocio antes de abrir la transacción minimiza bloqueos inútiles.



Buenas Prácticas

Utilizar THROW en lugar del obsoleto RAISERROR, ya que respeta el lote original y detiene la ejecución inmediatamente sin enmascarar el error subyacente.

```
BEGIN TRY
    -- Validación Senior (Fail-fast antes de la transacción)
    IF EXISTS (SELECT 1 FROM Academico.Matricula
               WHERE IdMatricula = @IdMatricula AND TieneDeuda = 1)
        THROW 50001, 'El estudiante posee deudas pendientes.', 1;

    BEGIN TRANSACTION;
    INSERT INTO Tramite.Tramite (IdMatricula, IdTipoTramite, FechaInicio)
        VALUES (@IdMatricula, @IdTipoTramite, CAST(SYSDATETIME() AS DATE));

    SET @IdTramite = SCOPE_IDENTITY();
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;

    INSERT INTO Tramite.LogErrores (ErrorNum, ErrorMsg, Linea)
    VALUES (ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE());

    THROW; -- Propagar error al cliente
END CATCH
```


Misión: Tema 4, Proyecto 3

Diseñar un sistema que monitoree el crecimiento del log de transacciones y genere una alerta crítica al superar el 80% de su tamaño.

Justificación

Un archivo de Log lleno detiene por completo todas las operaciones de escritura (Inserts/Updates). Monitorear la tendencia de crecimiento previene caídas catastróficas del servicio.

Buenas Prácticas

Jamás usar AUTOGROW ilimitado en pequeños porcentajes. Pre-asignar el tamaño del log evita la fragmentación física extrema (Virtual Log Files - VLF).

LOG SPACE CAPACITY



```
DECLARE @LogUsage TABLE (  
    DBName SYSNAME, LogSize FLOAT, LogUsedPerc FLOAT, Status INT  
);  
  
INSERT INTO @LogUsage EXEC('DBCC SQLPERF(LOGSPACE)');  
  
IF EXISTS (SELECT 1 FROM @LogUsage  
    WHERE DBName = 'GradosTitulos' AND LogUsedPerc > 80)  
BEGIN  
    EXEC msdb.dbo.sp_send_dbmail  
        @profile_name = 'Perfil_UPLA',  
        @recipients = 'admin@upla.edu.pe',  
        @subject = 'CRÍTICO: Log de Transacciones > 80%',  
        @body = 'El log de GradosTitulos ha superado el umbral  
                seguro.';  
END
```


Misión: Tema 4, Proyectos 4 y 5

Registrar el tiempo de ejecución (CPU) de los SPs para generar un ranking, e implementar un marco integral de monitoreo.



Justificación

Analizar DMVs es el método más ligero para identificar cuellos de botella en CPU en producción sin el overhead de trazas extensas.



Buenas Prácticas

- Bloqueos: Monitorear sys.dm_os_waiting_tasks.
- Consultas Lentas: Habilitar Query Store para capturar regresiones históricas de planes de ejecución.

-- Ranking Dinámico de Consumo de CPU mediante DMVs

```
SELECT TOP 10
    OBJECT_NAME(object_id, database_id) AS SP_Name,
    avg_worker_time / 1000 AS [Avg CPU ms],
    execution_count,
    last_execution_time
FROM sys.dm_exec_procedure_stats
WHERE database_id = DB_ID('GradosTitulos')
ORDER BY [Avg CPU ms] DESC;
```

-- Monitoreo Activo de Bloqueos (Blocking)

```
SELECT session_id, wait_duration_ms, wait_type,
    blocking_session_id
FROM sys.dm_os_waiting_tasks
WHERE blocking_session_id <> 0;
```


Leaderboard: Matriz de Automatización Final

Proceso Automatizado	Herramienta	Frecuencia Sugerida
Backup Full + ZIP	PowerShell + Task Scheduler	Diario (23:00)
Reporte de Trámites Registrados	SQL Agent + T-SQL	Diario (08:00)
Plan de Mantenimiento Inteligente	T-SQL + Cursor + DMV	Semanal (Domingos 02:00)
Monitoreo de Crecimiento de Log	SQL Agent + Alerta	Cada 15 minutos
Ranking de Performance de SPs	DMV Analysis	Mensual

Gobernanza Fortalecida: Eliminación de tareas manuales, auditorías 100% transparentes y reducción del riesgo de pérdida de datos en un 95%.